

GAINE - Tema 1 - Fundamentos de bases de datos NoSQL

¿Qué es NoSQL?

Bibliografía:

- Básico
 - Materiales preparados por el Equipo Docente
 - [Akhtar2018] Cap. 4.

Conceptos básicos de bases de datos)

El **acrónimo CRUD** (*Create, Read, Update, Delete*) se refiere las consultas básicas de creación, lectura, actualización y borrado de registros.

Una base de datos es **escalable verticalmente** si puede crecer aumentando las capacidades (más memoria, un procesador más rápido, etc.).

Una base de datos es **escalable horizontalmente** si puede crecer añadiendo nuevas instancias a las ya existentes.

Bases de datos relacionales)

El **acrónimo ACID** define las características que cumplen las bases de datos relacionales:

- Atomicidad (*atomicity*)
- Consistencia (*consistency*)
- Aislamiento (*isolation*)
- Durabilidad (*durability*)

Atomicidad (*atomicity*) supone que cada transacción o bien se ejecuta, o bien no se ejecuta, pero nunca se queda en un estado intermedio.

Consistencia (*consistency*), también llamada consistencia fuerte, indica que los datos son consistentes con las reglas de integridad de la base de datos.

Aislamiento (*isolation*), a veces referido como **aislamiento serializable**, indica que múltiples transacciones ejecutándose en paralelo no interfieren entre ellas.

Durabilidad (*durability*) refiere a que las transacciones completadas permanecerán en la base de datos.

Las bases de datos relacionales permiten un escalado vertical, es decir, para que crezca se requiere mayor capacidad del sistema (memoria, procesamiento, etc.), apostando por la centralización.

Las consultas a bases de datos relacionales pueden contener consultas complejas, como aquellas que tienen *joins*.

El esquema en las bases de datos relacionales es dinámico, forzando que se siga una estructura rígida.

Las base de datos relacionales permite un escalado vertical, es decir, añadiendo más capacidad y recursos al nodo central.

Bases de datos NoSQL)

El **acrónimo BASE**, por el contrario, define las características que cumplen las bases de datos NoSQL:

- Básicamente disponible (*basically available*)
- Estado suave (*soft-state*)
- Consistencia eventual (*eventually consistency*)

Básicamente disponible (*basically available*) indica que la base de datos se mantendrá en funcionamiento aunque algunos nodos no estén disponibles.

Estado suave (*soft-state*) indica que un mismo dato puede tener valores diferentes en distintas partes del sistema.

Consistencia eventual (*eventually consistency*) indica que al cabo de un tiempo la base de datos puede llegar a un estado consistente.

EXAM=(2025J2.1)

Las base de datos NoSQL permite un escalado horizontal, es decir, permite el crecimiento mediante la creación de nuevas instancias del sistema, apostando por la descentralización.

Las consultas a bases de datos NoSQL se suelen limitar a consultas básicas tipo CRUD .

El esquema en las bases de datos NoSQL es dinámico, permitiendo más flexibilidad.

Comparativa entre bases de datos relacionales y NoSQL:

	Relacional	NoSQL
Características	ACID	BASE
Forma de almacenamiento	Tabular	Clave-valor, documental, columnar, grafal
Esquema	Estático	Dinámico
Crecimiento	Vertical	Horizontal
Complejidad de consultas	Complejas	Solo CRUD

Teorema de CAP

EXAM=(2022J2.01, 2022J2.12, 2022S0.01, 2022S0.11, 2024J2.12, 2024S0.01, 2025J2.05)

Bibliografía:

- Materiales preparados por el Equipo Docente
- [Akhtar2018] Cap. 4.

El **teorema CAP** sostiene que todo sistema de datos distribuido es incapaz de garantizar simultáneamente las siguientes propiedades:

- **Consistencia** (*Consistency*): Se refiere a la uniformidad de los datos. Garantiza que todos los usuarios ven la misma información al mismo tiempo, sin importar a qué nodo se conecten. Si alguien actualiza un dato, ese cambio es visible para todos de inmediato.
- **Disponibilidad** (*Availability*): Es la capacidad de respuesta. Asegura que el sistema siempre te dé una respuesta (aunque no sea la versión más reciente del dato) cada vez que hagas una solicitud, sin importar si algunos componentes internos están fallando.
- **Tolerancia al particionamiento** (*Partition Tolerance*): Es la resistencia a fallos de comunicación. El sistema debe seguir operando aunque la red se fragmente, dejando a grupos de servidores aislados y sin posibilidad de hablar entre ellos.

En el mundo real, las redes no son perfectas, por lo que la tolerancia al particionamiento no es opcional en las redes distribuidas; es una necesidad. Esto obliga a elegir entre CP y AP en bases de datos distribuidas.

- **CA (Consistencia + Disponibilidad)**: El sistema prescinde de la distribubilidad antes que quedar desactualizada o no estar disponible. Ejemplo: MariaDB, MySQL.
- **CP (Consistencia + Partición)**: El sistema prefiere bloquearse o dar error antes que devolver un dato desactualizado durante un fallo de red. Ejemplo: MongoDB, Neo4j.
- **AP (Disponibilidad + Partición)**: El sistema sigue respondiendo siempre, aunque corra el riesgo de entregar datos que aún no se han sincronizado en todos los nodos. Ejemplo: CouchDB, Cassandra.

Cada base de datos NoSQL estudiada en la asignatura sigue una preferencia distinta:

Base de datos	CAP por defecto	CAP alternativo
MariaDB	CA	CP
Redis	CP	
MongoDB	CP	

Base de datos	CAP por defecto	CAP alternativo
Cassandra	AP	CA
NeoSQL	CP	

Modelos de datos

Bibliografía:

- Materiales preparados por el Equipo Docente
- [Akhtar2018] Cap. 4.

Los modelos de bases de datos NoSQL son:

- Clave-valor
- Documental
- Columnar
- Grafo

Diferencias entre bases de datos NoSQL)

Aspectos en los que puede haber diferencias entre bases de datos NoSQL:

- Arquitectura (maestro-esclavo vs. masterless)
- Modelos de distribución de datos
- Modelo de desarrollo
 - Complejidad de las consultas

Sentencias equivalentes a SQL en NoSQL vistos en la asignatura:

SQL (relacional)	Redis	MQL (MongoDB)	CQL (Cassandra)	Cypher (Neo4j)
SELECT *	SORT	aggregate(),	SELECT *	MATCH (n)
FROM tabla		find()	FROM tabla	RETURN n
SELECT	HGETALL (en	find({},	SELECT	RETURN
col1, col2	hash)	{col1},	col1, col2	n.prop1,
		{col2:1}),		n.prop2
		aggregate({\$project})		
SELECT ...	N/A	aggregate({\$match: {	SELECT ...	MATCH ...
WHERE		condition})	WHERE	WHERE
INSERT	SET, HSET,	insertOne(),	INSERT INTO	CREATE,
	LPUSH/RPUSH,	insertMany()	... VALUES	MERGE
	SADD, ZADD			
UPDATE	SET, HSET,	updateOne(),	UPDATE ...	MATCH ...
	LPUSH/RPUSH,	updateMany()	SET	SET
	SADD, ZADD			

SQL (relacional)	Redis	MQL (MongoDB)	CQL (Cassandra)	Cypher (Neo4j)
DELETE	DEL, HDEL	deleteOne() / deleteMany()	DELETE	MATCH ... DELETE

Elección base de datos NoSQL)

EXAM=(2022J2.2,2023J2.6)

El siguiente pseudocódigo simplifica la elección de una base de datos NoSQL:

```

si datos altamente relacionados
  grafo
si no, si consultas de agregación
  columnar
si no, si consultas basadas en valores
  documental
si no
  clave-valor
finsi

```

Aplicaciones típicas de bases de datos:

- Relacional
 - Finanzas
 - Datos estructurados complejos
- Clave-valor
 - Caché
 - Sesiones
 - Carrito de la compra
- Documental
 - Gestión de contenido
 - Catálogos de productos
 - Analíticas ágiles
- Columnar
 - Logs
 - Historiales masivos
 - IoT
- Grafal
 - Recomendadores
 - Relaciones complejas interconectadas

Modelos de distribución

Bibliografía:

- Materiales preparados por el Equipo Docente
- [Sadalage12] Cap. 4.

Modelos de distribución:

- Replicación
 - Maestro-esclavo
 - Peer-to-peer
- Sharding

Replicación

Replicación es un modelo de distribución de datos en el que cada unidad de datos se replica entre distintos nodos del sistema.

Existen dos tipos de replicación:

- Maestro-esclavo / Primario-secundario
- Peer-to-peer (P2P)

EXAM=(2023J2.1)

En **maestro-esclavo** o **primario-secundario** los maestros aparecen una única vez para un dato, mientras que puede haber muchos esclavos. Los maestros sirven para leer y escribir, y los esclavos solo para escribir.

Si falla un máster, normalmente se pierde la capacidad de escribir en ese shard, pero pueden seguir permitiéndose lecturas desde sus réplicas esclavas

EXAM=(2023J2.2)

En **peer-to-peer (P2P)** todos los nodos son iguales, es decir, maestros. Si un nodo cae y hay shards con su contenido

En los exámenes, cuando se representa gráficamente una base de datos distribuida se suele usar esta convención:

- Triángulo: primario de un shard.
- Rectángulo: secundario/réplica de un shard.

Si hay dudas y no hay leyenda:

- Si hay un solo tipo de elemento, son todos primarios y es P2P
- Si hay más de un mismo tipo de elemento para un dato, tiene que ser una réplica en maestro-esclavo.

Sharding

Sharding es un modelo de distribución de datos en el que cada unidad de datos se distribuye entre distintos nodos del sistema, ya sea de manera manual o automática. No aporta tolerancia a fallos.

Indexación

Bibliografía:

- Materiales preparados por el Equipo Docente
- [Ohene2012]

Licencia y atribución

License CC BY 4.0

Este trabajo está licenciado bajo la licencia **Creative Commons Atribución 4.0 Internacional**.

© 2026, Colaboradores de apuntes-muicd-uned.